

brewery

March 3, 2025

1 Example 2: The brewery problem

The following models are partially flexible: we assume that the resources are always malt, hops, and yeast.

1.1 A concrete model

```
[1]: import pyomo.environ as pyo

[2]: # Define the index set
Beer = {'Light', 'Dark', 'Ale', 'Premium'}

[3]: # Data
w_malt = {'Light':1, 'Dark':1, 'Ale':0, 'Premium':3}
w_hops = {'Light':2, 'Dark':1, 'Ale':2, 'Premium':2}
w_yeast = {'Light':1, 'Dark':1, 'Ale':1, 'Premium':4}
revenue = {'Light':6, 'Dark':5, 'Ale':3, 'Premium':7}
avail_malt = 50
avail_hops = 150
avail_yeast = 80

[4]: # Create a Concrete model
model = pyo.ConcreteModel()

[5]: # Build the decision variables
model.x = pyo.Var(Beer, within=pyo.NonNegativeReals)

[6]: # Build the objective function
def obj_value_rule(model):
    return sum(revenue[b] * model.x[b] for b in Beer)

model.obj = pyo.Objective(rule=obj_value_rule, sense = pyo.maximize)

[7]: # Build the constraints
model.malt = pyo.Constraint(expr = sum(w_malt[i]*model.x[i] for i in Beer) <=
    ↪ avail_malt)
model.hops = pyo.Constraint(expr = sum(w_hops[i]*model.x[i] for i in Beer) <=
    ↪ avail_hops)
```

```
model.yeast = pyo.Constraint(expr = sum(w_yeast[i]*model.x[i] for i in Beer) <=
↪avail_yeast)
```

```
[8]: # Solve the model through the GLPK solver
```

```
from pyomo.opt import SolverFactory
```

```
solver = SolverFactory('glpk')
```

```
results = solver.solve(model)
```

```
[9]: # Print the optimization results
```

```
print()
```

```
model.display() # List of all optimization results
```

```
print()
```

```
print('Optimal value: ', pyo.value(model.obj)) # Print the value of model.obj
```

```
↪(i.e., optimal objective value)
```

Model unknown

Variables:

```
x : Size=4, Index={Dark, Premium, Light, Ale}
```

```
Key      : Lower : Value : Upper : Fixed : Stale : Domain
```

```
Ale      :      0 :  30.0 :   None : False : False : NonNegativeReals
```

```
Dark     :      0 :  10.0 :   None : False : False : NonNegativeReals
```

```
Light    :      0 :  40.0 :   None : False : False : NonNegativeReals
```

```
Premium  :      0 :   0.0 :   None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
```

```
Key   : Active : Value
```

```
None  :   True : 380.0
```

Constraints:

```
malt : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None : 50.0 :  50.0
```

```
hops : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None : 150.0 : 150.0
```

```
yeast : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None : 80.0 :  80.0
```

Optimal value: 380.0

```
[10]: # Print the optimal solution
```

```
for i in Beer:
```

```
    print(i, pyo.value(model.x[i]))
```

Dark 10.0
Premium 0.0
Light 40.0
Ale 30.0

1.2 An abstract model

```
[11]: import pyomo.environ as pyo
```

```
[12]: # Create an Abstract model
model = pyo.AbstractModel()
```

```
[13]: # Build the index set
model.Beer = pyo.Set()
```

```
[14]: # Build parameters
model.w_malt = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.w_hops = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.w_yeast = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.revenue = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.avail_malt = pyo.Param(within=pyo.NonNegativeReals)
model.avail_hops = pyo.Param(within=pyo.NonNegativeReals)
model.avail_yeast = pyo.Param(within=pyo.NonNegativeReals)
```

```
[15]: # Build the decision variables
model.x = pyo.Var(model.Beer, within=pyo.NonNegativeReals)
```

```
[16]: # Build the objective function
def obj_value_rule(model):
    return sum(model.revenue[i]*model.x[i] for i in model.Beer)

model.obj = pyo.Objective(rule=obj_value_rule, sense = pyo.maximize)
```

```
[17]: # Build the malt constraint
def malt_rule(model):
    return sum(model.w_malt[i]*model.x[i] for i in model.Beer) <= model.
    ↪ avail_malt

model.malt = pyo.Constraint(rule=malt_rule)
```

```
[18]: # Build the hops constraint
def hops_rule(model):
    return sum(model.w_hops[i]*model.x[i] for i in model.Beer) <= model.
    ↪ avail_hops

model.hops = pyo.Constraint(rule=hops_rule)
```

```
[19]: # Build the yeast constraint
def yeast_rule(model):
    return sum(model.w_yeast[i] * model.x[i] for i in model.Beer) <= model.
    ↪avail_yeast

model.yeast = pyo.Constraint(rule=yeast_rule)

[20]: # Load data
instance = model.create_instance('brewery.dat')

[21]: # Solve the model through the GLPK solver
solver = pyo.SolverFactory('glpk')
solver.solve(instance)

[21]: {'Problem': [{'Name': 'unknown', 'Lower bound': 380.0, 'Upper bound': 380.0,
'Number of objectives': 1, 'Number of constraints': 3, 'Number of variables': 4,
'Number of nonzeros': 11, 'Sense': 'maximize'}], 'Solver': [{'Status': 'ok',
'Termination condition': 'optimal', 'Statistics': {'Branch and bound': {'Number
of bounded subproblems': 0, 'Number of created subproblems': 0}}, 'Error rc': 0,
'Time': 0.011029243469238281}], 'Solution': [OrderedDict({'number of solutions':
0, 'number of solutions displayed': 0})]}
```

```
[22]: # Print the results
instance.display()
print()
print('Optimal objective value: ', pyo.value(instance.obj))
```

Model unknown

Variables:

```
x : Size=4, Index=Beer
  Key      : Lower : Value : Upper : Fixed : Stale : Domain
  Ale      :      0 :  30.0 :   None : False : False : NonNegativeReals
  Dark     :      0 :  10.0 :   None : False : False : NonNegativeReals
  Light    :      0 :  40.0 :   None : False : False : NonNegativeReals
  Premium  :      0 :   0.0 :   None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
  Key : Active : Value
  None : True : 380.0
```

Constraints:

```
malt : Size=1
  Key : Lower : Body : Upper
  None : None : 50.0 : 50.0
hops : Size=1
  Key : Lower : Body : Upper
```

```
None : None : 150.0 : 150.0
yeast : Size=1
Key   : Lower : Body : Upper
None  : None  : 80.0  : 80.0
```

Optimal objective value: 380.0

```
[23]: # Print the optimal solution
      for i in instance.Beer:
          print(i, pyo.value(instance.x[i]))
```

```
Light 40.0
Dark 10.0
Ale 30.0
Premium 0.0
```